Introduction to programming

Labor09

Revision

- Sets
  - empty set: set()
  - functions
  - union, intersection, difference, symmetric difference
- Dictionaries
  - empty dictionary: dict(), {}
  - functions
  - access dictionary items

Homework 1
Every day an apple keeps the doctor away

- https://progcont.hu/progcont/100358/?pid=201544

Homework 1
Every day an apple keeps the doctor away

- We have all heard the saying in the title many times **fruit is an essential part of a healthy diet.**
- Write a program that reads, line by line from standard input, the **names of fruits** and the **names of children who like that fruit.** Each line has the following format:
- `fruit_name:child_name[,child_name]...`

- Both fruit names and children's names are **unique strings.** There are no duplicate entries such that the same child likes the same fruit more than once.
- Your program should print to standard output the names of the children in **lexicographically increasing order**, along with the **number of different fruits** each child likes, using the **exact format shown in the sample output.**

Homework 1
Every day an apple keeps the doctor away

Sample Input

```
apple:Pete,Kate
plum:John
banana:Kate,John,Charles
pineapple:Hannah,Anna,Penny
apricot:Fanny,Pete
```

Output for Sample Input

```
Anna: 1
Charles: 1
Fanny: 1
Hannah: 1
John: 2
Kate: 2
Penny: 1
Pete: 2
```

Homework 1
Every day an apple keeps the doctor away

```
import sys

children = {}
for line in sys.stdin:
    data = line.split(":")
    names = data[1].strip().split(",")
    for i in range(len(names)):
        if names[i] in children:
            children[names[i]] += 1
        else:
            children[names[i]] = 1

for key, value in sorted(children.items()):
    print(f"{key}: {value}")
```

Homework 2
New curriculum model

- https://progcont.hu/progcont/100278/?pid=201288

Homework 2
New curriculum model

- Write a program that reads an integer **N** from standard input. The next **N** lines each contain a subject code, a subject name, and a credit value in the following format:
- `Subject_code:subject_name:credit_number`
- The subject_code and subject_name are strings
- credit_number is a positive integer.
- The subject_code is always in the format "INBMM-xx-yy," where xx represents the recommended semester number, and yy is the unique identifier for the subject.
- Your program should print **only the subject names**, sorted in **ascending order** based on the recommended semester number. If multiple subjects belong to the same semester, they must be ordered by **increasing yy** (their unique identifier!

Homework 2
New curriculum model

- Sample Input:

```
5
INBMM-02-07:Data Structures and Algorithms:6
INBMM-01-01:Algorithms and Fundamentals of Programming:2
INBMM-01-02:Electronics:6
INBMM-03-14:Fundamentals of Economics:6
INBMM-02-08:Mathematics for Engineers 2:6
```

- Output for Sample Input:

```
Algorithms and Fundamentals of Programming
Electronics
Data Structures and Algorithms
Mathematics for Engineers 2
Fundamentals of Economics
```

Homework 2
New curriculum model

```
n = int(input())
subjects = {}

for i in range(n):
    data = input().split(":")
    subjects[data[0]] = data[1]

for key, value in sorted(subjects.items()):
    print(value)
```

Homework 3 – Club loyalty

- https://progcont.hu/progcont/100360/?pid=201553

Homework 3 – Club loyalty

- Some athletes remain loyal to a single club throughout their entire sports career. Others are not like that: they change clubs from time to time to advance their careers or to improve their financial situation.
- Write a program that reads an integer **K** from the first line of standard input. After that, the input contains data about sports clubs, one per line, **until end-of-file (EOF).**
- Each line contains the name of a club and the names of all athletes who have ever been associated with that club, in the following format:
- `club_name:athlete_name[,athlete_name]...`
- Your program should print the names of all athletes who have been associated with **more than K different clubs.**
- If there are multiple such athletes, order them
  - first by the **number of clubs** they have been associated with (in **descending** order – more clubs comes first),
  - and if there is still a tie, by their **names in lexicographical ascending order**.

The output format should match the sample output exactly.

Homework 3 – Club loyalty

- Sample Input
  - 2
  - Golden Team:Grosics,Buzanszky,Lorant,Lantos,Bozsik,Zakarias,Budai,Kocsis,Hidegkuti,Puskas,Czibor
  - Mighty Magyars:Buzanszky,Lantos,Zakarias,Hidegkuti
  - Marvellous Magyars:Grosics,Buzanszky,Bozsik,Hidegkuti,Puskas
  - Magnificent Magyars:Lorant,Budai,Kocsis,Hidegkuti,Puskas
  - Magical Magyars:Buzanszky,Lantos,Hidegkuti,Puskas
- Output for Sample Input
  - Hidegkuti: 5
  - Buzanszky: 4
  - Puskas: 4
  - Lantos: 3

```
import sys
n = int(input())
footballers = {}
for line in sys.stdin:
    data = line.split(":")
    names = data[1].strip().split(",")
    for i in range(len(names)):
        if names[i] in footballers:
            footballers[names[i]] += 1
        else:
            footballers[names[i]] = 1

for key, value in sorted(footballers.items(),
            key = lambda x: (-x[1], x[0])):
    if value > n:
        print(f"{key}: {value}")
```

Lambda functions

- A lambda function is a small anonymous function.
- It can take any number of arguments, but it can contain only one expression.

Syntax:

- lambda *arguments* : *expression*
- The expression is evaluated and its result is returned.

```
print((lambda x, y, z: (x + y + z) * 10)(1, 2, 3))
60

lista = ["Jabba", "Han", "Luke"]
print(sorted(lista, key = lambda item: -len(item)))
['Jabba', 'Luke', 'Han']
```

Command line arguments

- It is possible to pass values from the command line to a Python program when it is executed.
  These values are called **command-line arguments.**
- Command-line arguments are the values written **after the program's name** in the operating system's command-line shell.
- Python provides several ways to handle command-line arguments.
- The three most common are:
  - **Using sys.argv**
  - Using getopt module
  - Using argparse module

Command line arguments

```
argc - number of command line arguments -> int

sys.argv[0] - program name/file location -> str

sys.argv[1] - first command line argument -> str

sys.argv[2] - second command line argument -> str

...

sys.argv[n] - nth command line argument -> str
```

Command line arguments

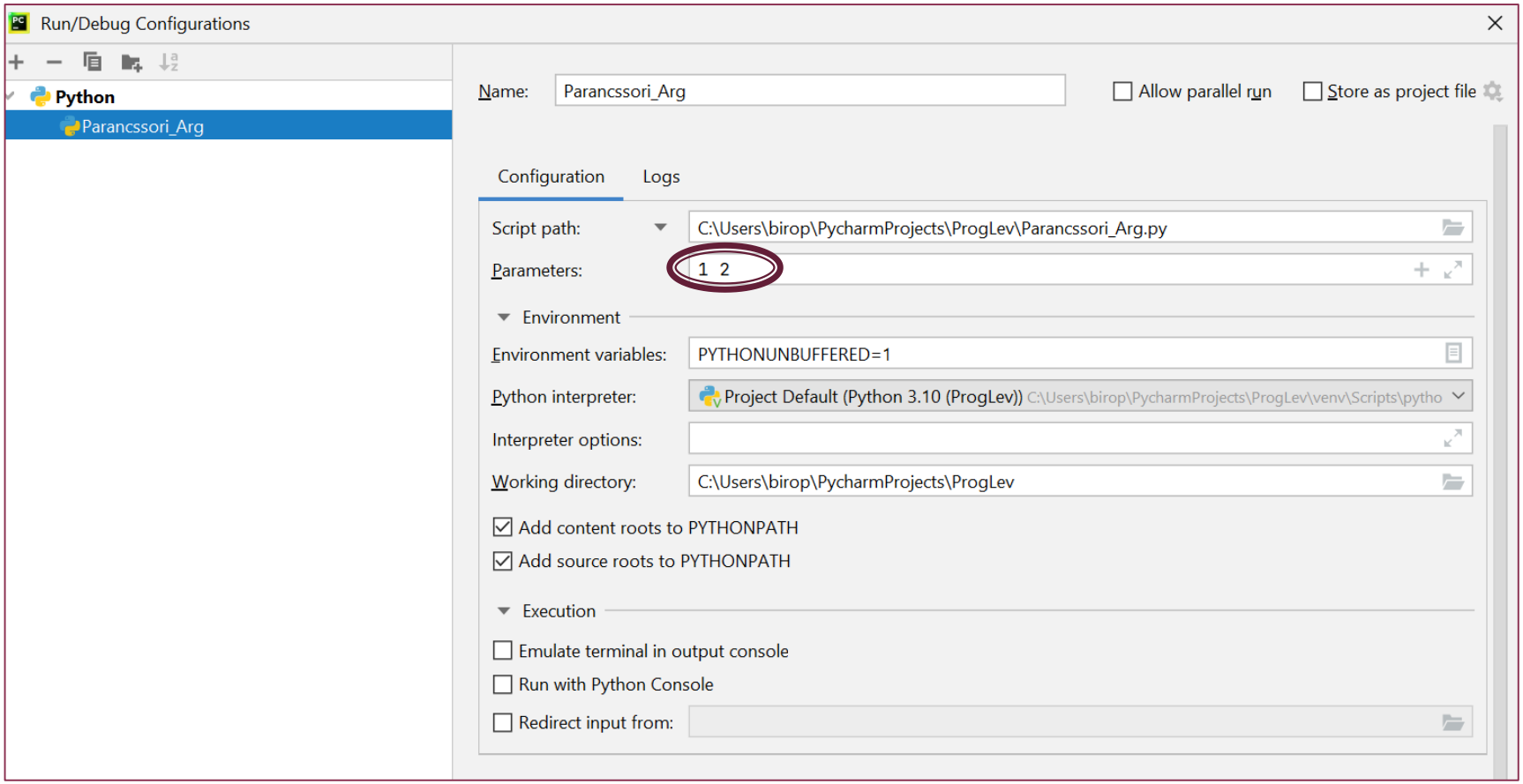
```
import sys

argc = len(sys.argv)
print("Number of arguments:", argc)

print("File location:", sys.argv[0])

print("Arguments:")
for i in range(1, argc):
    print(sys.argv[i])
```

Command line arguments



Command line arguments

RUN – Total Commander -> CMD:

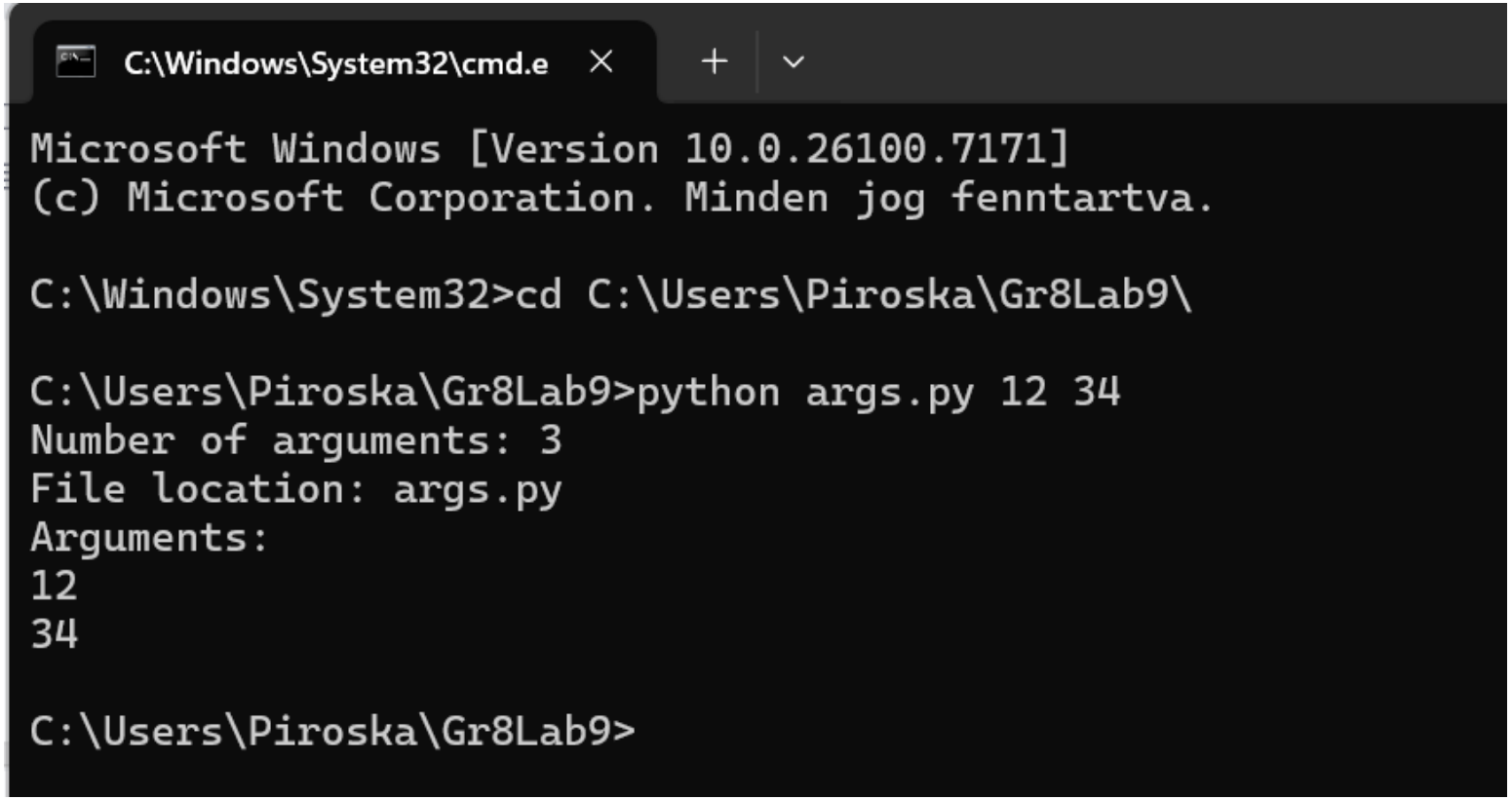
- c:/Users/PycharmProjects/Gr8Lab9/python arg.py 1 2

Output:

- Number of arguments: 3
- File locations:
  c:/Users/PycharmProjects/Gr8Lab9/arg.py
- Arguments:
- 1
- 2

Command line arguments

- CMD – Command Prompt



Command line arguments

- Write a Python program that reads numeric values from the **command-line arguments** and calculates the **product** of all the given numbers.
- Only the command-line arguments (not the program name) should be processed. Assume that all arguments are valid integers.

Command line arguments

```
import sys

argc = len(sys.argv)
print("Number of arguments:", argc)

p = 1
for i in range(1, argc):
    p = p * int(sys.argv[i])
print("Product = ", p)
```

Number of odd elements

The function

- Write a function named **count_of_odds** that returns the number of **odd numbers** found in the command-line arguments.

Returned value

- The function should return **the number of arguments that satisfy the condition (i.e., the number of odd integers).**
  https://viskillz.inf.unideb.hu/prog/#/?exercise=P108101c&page=sheet&week=P1081

Number of odd elements

```
import sys

def count_of_odds() -> int:
    count = 0
    for number in sys.argv[1:]:
        if int(number) % 2 == 1:
            count += 1
    return count

def main():
    print(count_of_odds())

if __name__ == "__main__":
    main()
```

Number of odd elements

```
import sys

def count_of_odds(numbers: list[int]) -> int:
    count = 0
    for number in numbers:
        if number % 2 == 1:
            count += 1
    return count

def main():
    numbers = []
    argc = len(sys.argv)
    for i in range(1, argc):
        numbers.append(int(sys.argv[i]))

    print(count_of_odds(numbers))

if __name__ == "__main__":
    main()
```

Number of prime elements

The function

- Write a function named count_of_that returns the number of **prime numbers** found in the command-line arguments.

Returned value

- The function should return **the number of arguments that satisfy the condition**

https://viskillz.inf.unideb.hu/prog/#/?exercise=P108103c&page=sheet&week=P1081

Number of prime elements

```
import sys
import math

def isprim(n: int) -> bool:
    if n == 0 or n == 1:
        return False
    d = 2
    while d <= math.sqrt(n):
        if n % d == 0:
            return False
        d += 1
    return True
```

```
def count_of_primes() -> int:
    count = 0
    for number in sys.argv[1:]:
        if is_prime(int(number)):
            count += 1
    return count

def main():
    print(count_of_primes())

if __name__ == '__main__':
    main()
```

Number of prime elements

```python
import sys
import math

def isprim(n: int) -> bool:
    if n == 0 or n == 1:
        return False
    d = 2
    while d <= math.sqrt(n):
        if n % d == 0:
            return False
        d += 1
    return True
```

```python
def count_of_primes(numbers: list[int]) ->
int:
    count = 0
    for number in numbers:
        if isprime(number):
            count += 1
    return count

def main():
    numbers = []
    argc = len(sys.argv)
    for i in range(1, argc):
        numbers.append(int(sys.argv[i]))

    print(count_of_primes(numbers))

if __name__ == "__main__":
    main()
```

File Handling

- **Open a file**
- To work with files in Python, you can use the **open()** function, which returns a **file object.**
- **It takes the name of the file to open as the first parameter.**
  - **As a second parameter, you can specify how to open the file:**
    - read: `'r'`,
    - write: `'w'`,
    - append: `'a'`.
    - (The default value is read, i.e. 'r'.)

```
file = open("test.txt", 'r')
```

File Handling

- **Open a file**
  - To open a file for reading it is enough to specify the name of the file:

```
file = open("test.txt")
```

  - If the file is in a different location, you will have to specify the file path, like this:

```
file = open("c:/IntroProg/test.txt")
```

Read data from a file

Read the entire contents of the file:

```
with open('test.txt') as file:
    lines = file.read()
    print(lines)
```

Read data from a file

Read the first line of the file:

```
with open("test.txt") as file:
    line = file.readline().strip("\n")
    print(line)
```

Read data from a file

Read all lines of the file,
the result is a list of strings:

```
with open("test.txt") as file:
    lines = file.readlines()
    print(lines)
```

Read data from a file

Read the file line by line:

```
with open("test.txt") as file:
      for line in file:
         print(line.strip("\n"))
```

File Handling

- **Create a file**

- Writing to a file is done using the file object's `write()` function, which behaves similarly to the print() function.

- However, note that `write()` **accepts only a single string argument**.
- Therefore, if you want to write multiple values on the same line, you must use **string concatenation** or **string formatting**.

Writing to a file

Writing to a file – one line:

```
with open("out.txt", "w") as file:
    file.write("apple\n")
```

Writing to a file

Writing to a file – more line:

```
with open("out.txt", "w") as file:
    file.write("apple\n")
    file.writelines(["pear\n", "orange\n"])
```

Exceptions

```
try:
    with open("out3.txt") as file:
        for line in file:
            print(line.strip("\n"))
except FileNotFoundError:
    print("File not found!")
```

Greatest common divisor

- https://progcont.hu/progcont/100132/?pid=200801

Greatest common divisor

- Write a program that reads **positive integers** from the text file specified as the **first command-line argument**.
- Each line of the file may contain **multiple integers**, separated by spaces.
- For **each line** of input, the program should print to standard output the **greatest common divisor (GCD)** of all numbers on that line!

Greatest common divisor

- Sample Input

sample.txt -> Command Line Argument

```
60 24 72
8
15 25 35
```

- Output for Sample Input

```
12
8
5
```

Solution

```
import sys

def gcd(a: int, b: int) -> int:
    while a != b:
        if a > b:
            a = a - b
        else:
            b = b - a
    return a

def main():
    with open(sys.argv[1]) as file:
        for line in file:
            numbers = line.strip("\n").split(" ")
            num = int(numbers[0])
            for i in range(1, len(numbers)):
                num = gcd(num, int(numbers[i]))
            print(num)
if __name__ == "__main__":
    main()
```

City Street Length Aggregation

- https://progcont.hu/progcont/100133/?pid=200802

City Street Length Aggregation

- Write a program that takes the name of a text file as its **first command-line argument**.
  The text file contains data about cities, one per line, in the following format:
- `city_name;street_name;street_length;bus_stops_number`
- Your program should **sum the lengths of all streets** for each city, then print the **names of the cities** to standard output, **one per line**, in **decreasing order** based on their total street length.
- If multiple cities have the same total street length, they must be listed in **lexicographical ascending order**.

City Street Length Aggregation

- **Sample Input**

sample.txt -> **Command line argument**

  - `Debrecen;Piac utca;5.5;4`
  - `Budapest;Bartók Béla út;8.0;12`
  - `Debrecen;Bartók Béla út;6.2;7`
  - `Szeged;Zákány utca;4.2;0`

- **Output for Sample Input**
  - `Debrecen`
  - `Budapest`
  - `Szeged`

Solution

```
import sys
streets={}

with open(sys.argv[1]) as file:
    for line in file:
        data = line.strip("\n").split(";")
        #print(data)
        if data[0] in streets:
            streets[data[0]]+=float(data[2])
        else:
            streets[data[0]]=float(data[2])
    #print(streets)
for key, value in sorted(streets.items(),
                         key=lambda x: (-x[1], x[0])):
    print(key)
```

Homework 1

Number of local maximums

- https://viskillz.inf.unideb.hu/prog/#/?week=P1081&exercise=P108109c&page=sheet

Homework 1

The function

- Write a function named `count_of_local_maximums` that returns the number of elements in the list (given as a parameter) that are **greater than both of their neighbours.**

Returned value

- The function should return **the number of list elements that satisfy this condition.**

Command Line Arguments:

- `12 34 12 56 5`

Output:

- 2

Homework 2 – Formula 1

- Write a program whose **first command-line argument** is the name of a text file.
- The text file contains data about Formula 1 drivers, one per line, in the following format:

- `driver_name;race_location;number_of_lap_taken;placing`

- Your program should **sum the total number of laps completed by each driver**, then print the **names of the drivers**, one per line, in **descending order** based on their total number of completed laps!
- If multiple drivers have completed the same number of laps, their names must be printed in **lexicographical (ascending) order.**

Homework 2

- Command Line Argument:

```
sample.txt
```

- Contents of the sample.txt file:

```
Lewis Hamilton;Melbourne;58;1
Daniel Ricciardo;Melbourne;57;6
Lewis Hamilton;Sepang;56;2
Fernando Alonso;Sepang;20;0
```

- The result of the run on standard output:

```
Lewis Hamilton
Daniel Ricciardo
Fernando Alonso
```

Homework 3

Prime selection

- https://progcont.hu/progcont/100317/?pid=201426

Homework 3 – Prime selection

- Write a program that takes the name of a text file as its **first command-line argument.**
- Each line of the file contains a **sequence of integers**, separated by **exactly one space**, and every line contains **at least one value.**
- For each line, your program should extract the **prime numbers**, sort them into an **increasing sequence**, and print them to standard output.
- If a line contains **no prime numbers**, your program must print the string: "NOTHING".
- See the sample output for the exact required format!

Homework 3 – Prime selection

- Command Line Argument:

```
sample.txt
```

- Contents of the sample.txt file:

```
1 2 3 4 5 6 7 8 9
25 13 12 10 21 53
3 5 7 3 5 7 3 5 7
2 4 8 16 32
```

- The result of the run on standard output:

```
2, 3, 5, 7
13, 53
3, 5, 7
2
```